

CSCE 763: Digital Image Processing

Homework #5

Instructor: Dr. Yan Tong
University of South Carolina
Solutions by: JC Vaught

Due: 1:15 pm EST, Thursday, April 9

Table of Contents	
Problem 1: Image Entropy and Compression (25 pts)	2
Problem 2: Arithmetic Decoding (25 pts)	8
Problem 3: Line Break Detection Masks (25 pts)	11
Problem 4: Sobel Mask Decomposition (25 pts)	13

Problem 1: Image Entropy and Compression (25 pts)

Consider a simple 4×16 , 8-bit image:

```

22  22  21  94  93  94  166  167  166  167  253  254  254  255  253  55
20  22  21  22  94  95  93  166  167  166  254  253  255  255  254  54
21  21  20  22  95  95  95  165  166  166  255  254  254  254  253  54
21  21  22  95  95  94  166  167  165  166  254  255  253  253  255  55

```

- Compute the entropy of the image. (5 pts)
- Compress the image using Huffman coding and report the compression ratio using Huffman coding. (10 pts)
- Use another data compression algorithm shown in the lecture to obtain a higher compression ratio. (10 pts)

Part (a): Compute Entropy

The image has $4 \times 16 = 64$ pixels. Counting the frequency of each distinct intensity value yields 14 unique symbols.

Value	Count	p_i	Value	Count	p_i
20	2	$2/64$	95	6	$6/64$
21	6	$6/64$	165	2	$2/64$
22	6	$6/64$	166	8	$8/64$
54	2	$2/64$	167	4	$4/64$
55	2	$2/64$	253	6	$6/64$
93	2	$2/64$	254	8	$8/64$
94	4	$4/64$	255	6	$6/64$

Group the symbols by probability to compute entropy $H = -\sum p_i \log_2 p_i$:

- 5 symbols with $p = 2/64$: each contributes $-\frac{2}{64} \log_2 \frac{2}{64} = \frac{5}{32}$. Subtotal: $5 \times \frac{5}{32} = \frac{25}{32} = 0.78125$.
- 5 symbols with $p = 6/64$: each contributes $-\frac{6}{64} \log_2 \frac{6}{64} = \frac{3}{32}(5 - \log_2 3)$. Subtotal: $5 \times 0.32016 = 1.60078$.
- 2 symbols with $p = 4/64$: each contributes $-\frac{4}{64} \log_2 \frac{4}{64} = \frac{4}{16} = 0.25$. Subtotal: $2 \times 0.25 = 0.50$.
- 2 symbols with $p = 8/64$: each contributes $-\frac{8}{64} \log_2 \frac{8}{64} = \frac{3}{8} = 0.375$. Subtotal: $2 \times 0.375 = 0.75$.

Results

$$H = 0.78125 + 1.60078 + 0.50 + 0.75 = 3.632 \text{ bits/pixel}$$

Part (b): Huffman Coding

Build a Huffman tree by repeatedly combining the two lowest-frequency symbols.

Huffman Tree Construction:

Sort symbols by frequency (ascending): 20(2), 54(2), 55(2), 93(2), 165(2), 94(4), 167(4), 21(6), 22(6), 95(6), 253(6), 255(6), 166(8), 254(8).

1. Combine 20(2) + 54(2) $\rightarrow N_1(4)$
2. Combine 55(2) + 93(2) $\rightarrow N_2(4)$
3. Combine 165(2) + $N_1(4) \rightarrow N_3(6)$
4. Combine $N_2(4)$ + 94(4) $\rightarrow N_4(8)$
5. Combine 167(4) + $N_3(6) \rightarrow N_5(10)$
6. Combine 21(6) + 22(6) $\rightarrow N_6(12)$
7. Combine 95(6) + 253(6) $\rightarrow N_7(12)$
8. Combine 255(6) + $N_4(8) \rightarrow N_8(14)$
9. Combine 166(8) + 254(8) $\rightarrow N_9(16)$
10. Combine $N_5(10)$ + $N_6(12) \rightarrow N_{10}(22)$
11. Combine $N_7(12)$ + $N_8(14) \rightarrow N_{11}(26)$
12. Combine $N_9(16)$ + $N_{10}(22) \rightarrow N_{12}(38)$
13. Combine $N_{11}(26)$ + $N_{12}(38) \rightarrow \text{Root}(64)$

The resulting Huffman code table is as follows.

Symbol	Count	Code	Bits	Total Bits
166	8	100	3	24
254	8	101	3	24
95	6	000	3	18
253	6	001	3	18
255	6	010	3	18
21	6	1110	4	24
22	6	1111	4	24
94	4	0111	4	16
167	4	1100	4	16
55	2	01100	5	10
93	2	01101	5	10
165	2	11010	5	10
20	2	110110	6	12
54	2	110111	6	12

Results

Average code length: $236/64 = 3.6875$ bits/pixel (close to entropy of 3.632).

Original image: $64 \times 8 = 512$ bits. Huffman-coded: 236 bits.

$$\text{Compression Ratio} = \frac{512}{236} \approx 2.169$$

Part (c): Lossless Predictive Coding (DPCM) + Huffman

Huffman coding on the raw pixel values ignores spatial correlation between neighboring pixels. A higher compression ratio can be achieved by first applying *differential pulse code modulation* (DPCM) to exploit the strong inter-pixel redundancy, then Huffman coding the prediction errors.

Prediction scheme: The first pixel is sent directly (8 bits). Each subsequent pixel in a row is predicted by its left neighbor. The first pixel of rows 2–4 is predicted by the pixel directly above it. The transmitted quantity is the prediction error $e_n = x_n - \hat{x}_n$.

Prediction errors (63 values):

Row 1 (cols 2–16): 0, −1, 73, −1, 1, 72, 1, −1, 1, 86, 1, 0, 1, −2, −198

Row 2 (col 1 from above, then cols 2–16):

−2, 2, −1, 1, 72, 1, −2, 73, 1, −1, 88, −1, 2, 0, −1, −200

Row 3: 1, 0, −1, 2, 73, 0, 0, 70, 1, 0, 89, −1, 0, 0, −1, −199

Row 4: 0, 0, 1, 73, 0, −1, 72, 1, −2, 1, 88, 1, −2, 0, 2, −200

Error frequency distribution and Huffman codes:

Error	Count	Code	Bits	Total Bits
1	14	01	2	28
0	13	00	2	26
−1	11	111	3	33
−2	5	1101	4	20
2	4	1010	4	16
73	4	1011	4	16
72	3	1000	4	12
88	2	10011	5	10
−200	2	11000	5	10
89	1	100100	6	6
−198	1	100101	6	6
−199	1	110010	6	6
70	1	1100110	7	7
86	1	1100111	7	7
Error bits:				203

Total bits = 8 (first pixel) + 203 (Huffman-coded errors) = 211 bits.

Results

The prediction errors have entropy $H_e \approx 3.09$ bits (vs. 3.63 for raw pixels), confirming that DPCM removes significant inter-pixel redundancy.

$$\text{Compression Ratio} = \frac{512}{211} \approx 2.427$$

This exceeds the Huffman-only ratio of 2.169 because predictive coding exploits the spatial correlation that symbol-by-symbol Huffman coding cannot.

Problem 2: Arithmetic Decoding (25 pts)

The arithmetic decoding process is the reverse of the encoding procedure. Decode the message 0.222355 given the coding model. Assume that “!” is the symbol indicating the end of message. Show the decoding process.

Symbol	Probability
A	0.25
B	0.15
C	0.1
D	0.3
E	0.1
!	0.1

Step 1: Establish Cumulative Probability Ranges

Assign cumulative subintervals to each symbol in the order given.

Symbol	Probability	Range
A	0.25	[0, 0.25)
B	0.15	[0.25, 0.40)
C	0.10	[0.40, 0.50)
D	0.30	[0.50, 0.80)
E	0.10	[0.80, 0.90)
!	0.10	[0.90, 1.00)

Step 2: Iterative Decoding

Start with interval $[low, high) = [0, 1)$ and code value $v = 0.222355$.

Iteration 1: $[low, high) = [0, 1)$, width = 1

Subintervals:

A: $[0, 0.25)$, B: $[0.25, 0.40)$, C: $[0.40, 0.50)$, D: $[0.50, 0.80)$, E: $[0.80, 0.90)$, !:
 $[0.90, 1.00)$

$v = 0.222355$ falls in $[0, 0.25) \Rightarrow$ **Output: A**

Update: low = 0, high = 0.25

Iteration 2: $[low, high) = [0, 0.25)$, width = 0.25

Subintervals:

A: $[0, 0.0625)$, B: $[0.0625, 0.1)$, C: $[0.1, 0.125)$, D: $[0.125, 0.2)$, E: $[0.2, 0.225)$, !:
 $[0.225, 0.25)$

$v = 0.222355$ falls in $[0.2, 0.225) \Rightarrow$ **Output: E**

Update: low = 0.2, high = 0.225

Iteration 3: $[low, high) = [0.2, 0.225)$, width = 0.025

Subintervals:

A: $[0.2, 0.20625)$, B: $[0.20625, 0.21)$, C: $[0.21, 0.2125)$, D: $[0.2125, 0.22)$
E: $[0.22, 0.2225)$, !: $[0.2225, 0.225)$

$v = 0.222355$ falls in $[0.22, 0.2225) \Rightarrow$ **Output: E**

Update: low = 0.22, high = 0.2225

Iteration 4: $[low, high) = [0.22, 0.2225)$, width = 0.0025

Subintervals:

A: $[0.22, 0.220625)$, B: $[0.220625, 0.221)$, C: $[0.221, 0.22125)$
D: $[0.22125, 0.222)$, E: $[0.222, 0.22225)$, !: $[0.22225, 0.2225)$

$v = 0.222355$ falls in $[0.22225, 0.2225) \Rightarrow$ **Output: !**

End-of-message symbol received. Stop decoding.

Verification: Encode “AEE!” to Confirm

Step	Symbol	New Low	New High
Init		0	1
1	A	$0 + 1 \times 0 = 0$	$0 + 1 \times 0.25 = 0.25$
2	E	$0 + 0.25 \times 0.8 = 0.2$	$0 + 0.25 \times 0.9 = 0.225$
3	E	$0.2 + 0.025 \times 0.8 = 0.22$	$0.2 + 0.025 \times 0.9 = 0.2225$
4	!	$0.22 + 0.0025 \times 0.9 = 0.22225$	$0.22 + 0.0025 \times 1.0 = 0.2225$

Final interval: $[0.22225, 0.2225)$. The code value $0.222355 \in [0.22225, 0.2225)$. ✓

Results

Decoded message: **A E E !**

Problem 3: Line Break Detection Masks (25 pts)

A binary image contains straight lines oriented horizontally, vertically, at 45° , and at -45° . Develop a set of 3×3 masks that can be used to detect 1-pixel breaks in these lines. For example, you may want to detect a pattern like 1 1 1 0 1 1 1 in the vertical direction. Assume that the intensities of the lines and background are represented by 1 and 0, respectively.

Step 1: Break Pattern Analysis

A 1-pixel break in a line means the center pixel is 0 (the gap) while the two nearest pixels along the line direction are 1 (the line continues on both sides). A good detection mask should produce a strong positive response at break locations and zero response at normal line points and background.

Consider the 3×3 neighborhood centered on the break pixel. The mask assigns positive weights to the positions where line pixels are expected (along the line direction, adjacent to the gap) and a negative weight at the center (which should be 0 for a break).

Step 2: Design the Four Masks

Horizontal line break — the line runs left-to-right, break at center:

$$\text{Pattern: } \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \quad \mathbf{M}_H = \begin{bmatrix} 0 & 0 & 0 \\ 1 & -2 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

Vertical line break — the line runs top-to-bottom, break at center:

$$\text{Pattern: } \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \quad \mathbf{M}_V = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

45° line break — the line runs from top-right to bottom-left, break at center:

$$\text{Pattern: } \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \quad \mathbf{M}_{45} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & -2 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

-45° line break — the line runs from top-left to bottom-right, break at center:

$$\text{Pattern: } \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \mathbf{M}_{-45} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 3: Response Verification

Each mask gives the following responses when centered on different pixel types.

Location	Center	Line Neighbors	Response
Break point	0	both = 1	$1 + (-2)(0) + 1 = \mathbf{2}$
Normal line	1	both = 1	$1 + (-2)(1) + 1 = \mathbf{0}$
Background	0	both = 0	$0 + (-2)(0) + 0 = \mathbf{0}$
Endpoint	1	1 and 0	$1 + (-2)(1) + 0 = \mathbf{-1}$

A threshold of > 0 (or specifically $= 2$) on the mask response cleanly isolates 1-pixel break locations. The masks produce the maximum response of 2 exclusively at break points, zero at normal line points and background, and -1 at endpoints.

Results

The four 3×3 break-detection masks are:

$$\mathbf{M}_H = \begin{bmatrix} 0 & 0 & 0 \\ 1 & -2 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{M}_V = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\mathbf{M}_{45} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & -2 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{M}_{-45} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

A response of 2 at any pixel indicates a 1-pixel break in the corresponding line orientation.

Problem 4: Sobel Mask Decomposition (25 pts)

The results obtained by a single pass through an image of some 2D masks can be achieved also by two passes using 1D masks for improving efficiency. Show that the response of Sobel masks

$$\mathbf{S}_x = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (1)$$

$$\mathbf{S}_y = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad (2)$$

can be implemented similarly by one pass of a vertical differencing mask

$$\mathbf{d}_v = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \quad (3)$$

followed by a horizontal smoothing mask

$$\mathbf{s}_h = \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} \quad (4)$$

Step 1: Separability of 2D Convolution

A 3×3 kernel $\mathbf{H}$ is *separable* if it can be expressed as the outer product of a column vector and a row vector:

$$\mathbf{H} = \mathbf{c} \cdot \mathbf{r}^T$$

When separable, the 2D convolution $f * \mathbf{H}$ can be decomposed into two sequential 1D convolutions:

$$f * \mathbf{H} = (f *_{\text{cols}} \mathbf{c}) *_{\text{rows}} \mathbf{r}$$

This reduces the computational cost from $O(m^2)$ multiplications per pixel (for an $m \times m$ kernel) to $O(2m)$.

Step 2: Decompose $\mathbf{S}_x$

Compute the outer product of $\mathbf{d}_v$ and $\mathbf{s}_h$:

$$\begin{aligned}\mathbf{d}_v \cdot \mathbf{s}_h &= \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} (-1)(1) & (-1)(2) & (-1)(1) \\ (0)(1) & (0)(2) & (0)(1) \\ (1)(1) & (1)(2) & (1)(1) \end{bmatrix} \\ &= \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} = \mathbf{S}_x \quad \checkmark\end{aligned}$$

Therefore, the 2D convolution with $\mathbf{S}_x$ can be computed by first convolving each column with $\mathbf{d}_v$ (vertical differencing), then convolving each row of the result with $\mathbf{s}_h$ (horizontal smoothing).

Step 3: Decompose $\mathbf{S}_y$

Define the complementary 1D masks: vertical smoothing $\mathbf{s}_v = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ and horizontal differencing $\mathbf{d}_h = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$.
Compute the outer product:

$$\begin{aligned}\mathbf{s}_v \cdot \mathbf{d}_h &= \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} (1)(-1) & (1)(0) & (1)(1) \\ (2)(-1) & (2)(0) & (2)(1) \\ (1)(-1) & (1)(0) & (1)(1) \end{bmatrix} \\ &= \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \mathbf{S}_y \quad \checkmark\end{aligned}$$

Therefore, $\mathbf{S}_y$ is computed by first convolving each column with $\mathbf{s}_v$ (vertical smoothing), then convolving each row with $\mathbf{d}_h$ (horizontal differencing).

Step 4: Equivalence of the Two-Pass Implementation

Both Sobel masks are separable products of the same two primitive 1D operations — a 3-point central difference $[-1, 0, 1]$ and a 3-point binomial smoothing $[1, 2, 1]$ — applied in perpendicular directions.

Sobel Mask	Pass 1 (columns)	Pass 2 (rows)
$\mathbf{S}_x$ (vertical edges)	$\mathbf{d}_v = [-1; 0; 1]$ (difference)	$\mathbf{s}_h = [1, 2, 1]$ (smooth)
$\mathbf{S}_y$ (horizontal edges)	$\mathbf{s}_v = [1; 2; 1]$ (smooth)	$\mathbf{d}_h = [-1, 0, 1]$ (difference)

Since convolution is commutative and associative, the order of the two 1D passes can also be swapped.

Results

Both Sobel masks are separable. Their responses are equivalent to two sequential 1D operations.

$$\mathbf{S}_x = \mathbf{d}_v \cdot \mathbf{s}_h^T = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} [1 \ 2 \ 1] \quad \mathbf{S}_y = \mathbf{s}_v \cdot \mathbf{d}_h^T = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} [-1 \ 0 \ 1]$$

This reduces the cost per pixel from 9 multiplications to 6 (two passes of 3 each).